Roma Tre

Università degli Studi

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche

Tesi di Laurea

Protecting the autonomic manager

A comparative analysis of Quorum and CometBFT as blockchain backends for Self-Protecting Systems

Corso di Laurea: Ingegneria Informatica

Candidato: Valerio Tatti

Matricola: 609824

Relatore: Prof. Stefano Iannucci

Anno Accademico: 2025/2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

In modern IT infrastructures, the rapid shift toward cloud-native, containerized microservices has dramatically expanded the digital attack surface. The escalation of sophisticated, high-velocity cyber threats and the ever-growing volume of sensitive data housed in these environments have driven the need for computing systems capable of defending themselves autonomously. Within this context, the autonomic computing paradigm has become a cornerstone of modern security engineering, aiming to reduce human operational burden and accelerate threat response. However, while automation resolves many latency and coordination challenges, it introduces new systemic vulnerabilities. If the automated security logic itself is targeted, it can become a vector for devastating privilege escalation and system-wide compromise.

1.1 The Paradigm of Self-Protecting Systems

Self-Protecting Systems [1] (SPS) are autonomic software systems designed to detect, analyze, and mitigate security threats dynamically with limited or no human intervention. Following the classic autonomic feedback loop, an SPS typically comprises two primary macro-components: a *managed system* and an *autonomic manager*. The managed system represents the production environment, services, or software applications exposed to potential attacks. Conversely, the autonomic manager serves as the central controller responsible for executing the security loop.

To achieve continuous protection, the autonomic manager organizes its operations into a pipeline governed by two main functional sub-components:

- **Intrusion Detection Systems (IDS):** Passive or active security sensors distributed across the managed system to monitor traffic, logs, or system calls and detect anomalous behavior or known attack signatures.
- **Intrusion Response Systems (IRS):** Decision-making engines responsible for selecting and executing the appropriate response actions (countermeasures)

to neutralize or mitigate the detected threats.

1.2 The Vulnerability of the Autonomic Manager

Although highly effective in mitigating threats directed at the managed system, this canonical design suffers from a critical structural vulnerability: the centralization of the autonomic manager. An adversary who penetrates the infrastructure can target the autonomic manager itself. By compromising this central controller, the attacker can silence intrusion alerts, disable response mechanisms, or even manipulate the manager into executing malicious countermeasures (e.g., disconnecting legitimate network segments or isolating critical database services).

Indeed, protecting the controller that protects the system remains an open and central challenge in autonomic security research. If the security logic is centralized and compromised, the integrity of the entire infrastructure is undermined, rendering the self-protecting features not only ineffective, but actively weaponized against the system.

1.3 Blockchain as a Mechanism of Trust

To eliminate the single point of failure represented by a centralized manager, our research [2] has proposed distributing the autonomic manager across a permissioned blockchain network. In this decentralized architecture, blockchain technology acts as a tamper-proof ledger and a distributed state machine (DSM) that logs security events and executes response logic securely through smart contracts.

This distributed approach establishes fundamental security guarantees that collectively safeguard the autonomic loop. To prevent a single compromised IDS sensor from injecting false alerts and triggering unauthorized countermeasures, the smart contract enforces a consensual voting mechanism. Under this scheme, updates to the shared security state are only committed once they receive corroborating reports from multiple independent, heterogeneous IDSs, ensuring that isolated sensor compromises do not lead to malicious actuations.

Also, by deploying the autonomic manager across a network of validator nodes running a Byzantine Fault-Tolerant (BFT) consensus protocol, the security loop gains strong structural resilience. At blockchain level, this mechanism tolerates the active compromise of up to 1/3 of validators while preserving ledger safety and finality under partial network compromise. This guarantee is distinct from IDS-side voting, which is the mechanism used to validate the semantic correctness of observations before they affect the consolidated monitored state.

Generally, the system is designed to reduce the attack surface as much as possible treating actuators as black boxes (blind), the actuators receive signed mitigation commands directly from the blockchain state, rather than querying the state of potentially compromised application containers directly.

1.4 Objectives and Contributions of the Thesis

While the theoretical integration of blockchain into the SPS architecture provides outstanding security guarantees, the practical choice of the underlying blockchain technology is not neutral. A blockchain-based autonomic manager has unique requirements: it must handle sparse, highly event-driven alert traffic with exceptionally low latency and high determinism. Conversely, many features native to public blockchains (such as virtual-machine-level gas metering, token economics, transaction fees, and dynamic peer discovery) add unnecessary complexity, expand the trusted computing base (TCB), and increase the attack surface.

The natural baseline choice for permissioned enterprise setups is GoQuorum (a fork of Ethereum). However, this thesis argues that general-purpose smart contract platforms incur severe, non-intrinsic performance overheads due to interpreted virtual machine (EVM) bytecode execution, rigid transaction fee mechanisms, and fixed-interval block production. As an alternative, this work explores CometBFT (the successor of Tendermint), a lower-level BFT consensus engine that supports highly optimized, application-specific distributed state machines.

The primary objective of this thesis is to present a comprehensive qualitative and empirical comparison between a conventional general-purpose platform (GoQuorum) and a requirement-driven, application-specific alternative (CometBFT) to determine which architecture best supports the performance and cyber-resilience demands of modern Self-Protecting Systems.

Specifically, the core contributions of this thesis are as follows:

1. **Requirement Formulation:** We define and classify a comprehensive set of functional and non-functional requirements (required, desirable, and not needed) for blockchain technology in the specific context of an autonomic SPS loop.

2. **Architectural Analysis:** We conduct a thorough qualitative comparison of GoQuorum and CometBFT, highlighting the architectural mismatch of general-purpose virtual machines versus application-specific state machines.

3. **Prototype Implementation:** We design and implement a realistic, fully containerized SPS prototype emulated via Kathará on top of Docker. The

prototype integrates three heterogeneous IDS engines (Snort, Suricata, and Zeek) protecting a vulnerable web application (OWASP Juice Shop).

4. **Deduplication Middleware:** We propose and develop a custom deduplication middleware layer for the member nodes. This layer filters repetitive alerts, preventing mempool saturation on the ledger keeping transaction number upper limited per block.

5. **Empirical Evaluation:** We perform extensive automated benchmarks under controlled attack conditions, comparing GoQuorum and CometBFT across key performance indicators, including end-to-end response latency, transaction throughput under heavy load, and CPU resource utilization.

1.5 Structure of the Thesis

To thoroughly explore the theoretical background, architectural design, and empirical evaluation of the proposed systems, the remainder of this thesis is structured as follows:

Chapter 2 – Technologies Setting and Related Work: Introduces the foundational concepts of autonomic computing, the MAPE cycle, Byzantine fault-tolerant consensus, the structural differences between public and permissioned blockchain systems, and reviews the state of the art in autonomic security.

Chapter 3 – A Cyber-Resilient Architectural Model: Details the technology-agnostic architecture of the Self-Protecting System, focusing on the interactions between IDSs, the smart contract state, and the blind actuators.

Chapter 4 – Requirement Alignment: Quorum vs. CometBFT: Formally identifies the blockchain properties strictly required for SPS applications and theoretically compares Quorum's general-purpose execution environment with CometBFT's application-specific design.

Chapter 5 – Implementation Details: Describes the prototype implementation, the containerized network topology emulated via Kathará, and the custom middleware developed for the evaluation.

Chapter 6 – Experiments and Results: Analyzes the performance of the two blockchain backends under varying workloads, presenting metrics on end-to-end latency, transaction throughput, and resource consumption.

Chapter 7 – Conclusions and Future Work: Summarizes the findings of the comparative analysis, acknowledges current limitations, and outlines directions for future research in decentralized system protection.

Chapter 2

Technologies Setting and Related Works

Before diving into the specific idea and implementation behind the system described in this research, it is necessary to introduce the basic theoretical concepts that inspired this architecture. These concepts will later provide us with the tools to comprehend by which metrics a blockchain-based autonomic agent should be measured, which paradigm first described autonomic systems, which security issues we can expect, how the current state-of-the-art (SOTA) solutions address them, and how our architecture compares to them.

2.1 Autonomic Computing and the MAPE Cycle

The paradigm of Autonomic Computing formally emerged in October 2001, when IBM released a manifesto identifying an upcoming software complexity crisis as the primary obstacle to further progress in the IT industry [3]. As computing environments began to weigh in at millions of lines of code, the expertise required for installation, configuration, and maintenance approached a limit on human capability to train new professionals. This challenge was exacerbated by the necessity of integrating several heterogeneous environments into corporate-wide systems and extending them globally across the Internet. IBM observed that relying solely on innovations in programming methods would not suffice to manage such massive and complex systems, which often require timely responses to rapid streams of changing demands.

To address this problem, IBM proposed the vision of autonomic computing: systems capable of managing themselves based on high-level objectives from human administrators [3]. The term "autonomic" was chosen for its biological connotation, tracing an analogy to the human autonomic nervous system. Just as the biological system governs vital, low-level functions such as heart rate and body temperature without conscious effort, autonomic computing aims to free human administrators from the burden of managing low-level system operations through

automation.

2.1.1 The Four Aspects of Self-Management

The essence of autonomic computing is self-management, intended to provide software that can adjust its behaviour in accordance with human objectives without their intervention [3]. IBM characterizes self-management through four fundamental aspects, often referred to as the "self-*" properties. Those principles have since materialized into concrete industry standards and architectural paradigms that govern contemporary network infrastructures:

- **Self-configuration**: Systems must automatically configure components and integrate themselves into complex environments following high-level policies. New components should be able to incorporate seamlessly, treating other components as black boxes. This modular perspective prefigured modern containerization paradigms, where container runtimes such as Docker have become the industry standard for application deployment.

- **Self-optimization**: Autonomic systems continually seek opportunities to improve their own performance and efficiency. In environments like complex middleware or databases that possess hundreds of manually set parameters, the system must monitor, experiment with, and tune these parameters dynamically. A notable industrial implementation of this concept is cloud-based auto-scaling (e.g., Amazon Web Services Auto Scaling), which dynamically adjusts resource provisioning to match fluctuating performance demands.

- **Self-healing**: The system is designed to automatically detect, diagnose, and repair localized software and hardware problems. Using knowledge of the system configuration and logs, the autonomic manager can identify root causes of failures, match them against known patches, and retest the system without manual debugging. In contemporary orchestration systems, self-healing is commonly implemented via automated health checks and liveness probes (e.g., in Kubernetes), which automatically restart or replace failing service instances.

- **Self-protection**: Autonomic systems defend the infrastructure as a whole against malicious attacks [4]. They use sensor reports to anticipate problems and take proactive steps to avoid or mitigate systemwide failures. An example of such proactive defense is represented by modern Web Application Firewalls (WAFs), such as Cloudflare's edge security systems, which autonomously detect, analyze, and block malicious traffic patterns.

2.1.2 The Structure of Autonomic Elements

Architecturally, autonomic systems are composed of elements that manage their own internal behavior and relationships with others in accordance with established policies [3]. An autonomic element typically consists of one or more managed elements coupled with a single autonomic manager.

- **Managed Element**: This component can be found in non-autonomic systems and can be a hardware resource (such as storage or a CPU) or a software resource (such as a database or a web service).
- **Autonomic Manager**: The manager distinguishes the autonomic element by monitoring the managed element and its environment, and executing actions based on the analysis of their information.

The internal logic of the autonomic manager is structured around a fundamental control loop known as the **MAPE-K** cycle [3, 5]:

1. **Monitor**: Collects, aggregates, and filters information from the managed element and its environment.
2. **Analyze**: Matches a diagnosis on the system's high-level status based on monitored information and triggers a response.
3. **Plan**: Creates or selects a sequence of actions that matches the organization's objectives.
4. **Execute**: Applies the derived plan / action to the managed element through effectors.
5. **Knowledge**: A central repository for policies, historical data, and system configurations that supports the other four phases defining the organization's objectives.

2.1.3 Security Challenges

Autonomic elements, unlike typical Web services, do not honor requests unconditionally, they provide services only if doing so is consistent with their internal goals [3].

This high degree of autonomy introduces critical system-level issues that cannot be ignored when evaluating attack surfaces. Because these systems rely on high-level goals, they are vulnerable to new forms of attack where an adversary might manipulate policies to gain systemic leverage [6]. This highlights the necessity for a secure-by-design management infrastructure that can address the "Who watches the watchers?" dilemma of autonomic elements [6, 7].

2.2 Blockchain in Permissioned Environments

The integration of blockchain technology to register system state represents an innovative approach to ensuring cyber-resilience [2]. However, in this specific application context, the choice of the underlying blockchain platform is not neutral, and it is necessary to define which blockchain properties are beneficial for a Self-Protecting System. A fundamental distinction must be made between public (permissionless) blockchains and private (permissioned) blockchains [8].

2.2.1 Smart Contracts and Response Logic

Before analyzing network access models, it is essential to define the mechanism through which the blockchain enforces security policies: the *smart contract*. A smart contract is a self-executing distributed software deployed on a blockchain that automatically enforces predefined rules and agreements between nodes without the need for intermediaries [9].

Once deployed, smart contracts execute deterministically across all nodes in the network, enabling transparent and immutable execution without requiring trust in a central authority [9]. In the context of a Self-Protecting System (SPS), the integrity of the smart contract's state and its execution is guaranteed by the inherent properties of the blockchain itself [2]. This enables the autonomic manager to maintain a consistent, system-wide view and coordinate responses even under partial compromise [2].

2.2.2 Characteristics of Permissioned Environments

Public blockchains are designed as open networks where anyone can participate. In contrast, a permissioned environment requires nodes to authenticate each other, often within a fixed trust boundary [8]. This architectural difference fundamentally alters the threat model and the system's performance requirements.

In a permissioned network, many attacks traditionally critical for permissionless blockchains, such Eclipse attack, are naturally mitigated because participant identities are controlled and membership is typically static [2].

Since permissionless environments adopt a fundamentally "trustless" model, they must employ strict, and often resource-expensive, consensus algorithms. In a permissioned blockchain, consensus can be simplified because all participating nodes are pre-vetted, identified, and operate within a predefined deployment. Consequently, the economic dynamics, gas fees, token incentives, and game-theory mechanisms typical of public cryptocurrencies designed to coordinate anonymous actors become functionally obsolete [10]. Furthermore, general-purpose virtual machines (VMs) such as the Ethereum Virtual Machine (EVM)

often introduce unnecessary complexity, processing overhead, and a broader attack surface when the system only requires a constrained, deterministic runtime for state transitions [10].

Instead, the focus in such environments shifts towards a specific set of essential properties. As highlighted in recent literature evaluating blockchain implementations for Self-Protecting Systems [10], the required features are:

- **Faul-Tolerant consensus with strong finality**: To tolerate a bounded number of compromised nodes and commit updates deterministically.
- **Static permissioned membership**: To match the fixed trust boundary of the deployment.
- **Cryptographic transaction signing**: To ensure the authenticity and accountability of alerts submitted by IDS's associated member nodes.

Additionally, desirable properties include in-memory replicated state handling, low and predictable latency to guarantee timely countermeasures, and push-based notifications to immediately trigger actuators upon state updates [10].

2.2.3 Alignment with SPS Requirements

For an SPS, the blockchain's sole functions are:

- To gather a shared state from Intrusion Detection Systems (IDS) [2].
- To ensure the reliable functioning of the response logic even in the presence of security faults in the managed system or in the autonomic manager [2].
- To communicate the response logic's decisions to the actuators securely [2].

Under these assumptions, permissioned environments offer indispensable advantages, such as static membership that matches the trust boundaries of a real-world deployment [10]. Furthermore, cryptographic transaction signing ensures the authenticity and accountability of sensor submissions [2, 10]. Finally, while blockchain provides an immutable audit trail for forensics, permissioned settings may allow for configurable data-retention policies, which is particularly relevant for resource-constrained or embedded systems [10].

Ultimately, the relevant blockchain properties in this domain are not openness or tokenization, but minimality, determinism, and resilience [10].

2.3 Distributed Consensus Algorithms

Blockchain technologies, despite their significant advantages, introduce substantial technological and implementation challenges that must be addressed to render

them practically useful. Consequently, every architectural decision must be evaluated carefully. Because a blockchain is fundamentally a distributed system, it requires a robust protocol to maintain network communication and ensure the reliability of the network's collective decisions. Distributed consensus algorithms address exactly this fundamental question: what is the protocol by which information is validated and permanently recorded as "official" within the distributed ledger? The selection of a consensus algorithm is by no means context-agnostic, on the contrary, aligning the consensus mechanism with the operational threat model is the primary driver of our architectural choices for SPS.

2.3.1 Proof-of-X: What Can Untrusted Environments Teach Us?

As previously discussed in Section 2.2, permissionless consensus mechanisms such as Proof-of-Work (PoW) and Proof-of-Stake (PoS) are unsuitable for our specific deployment due to their reliance on tokenomics, high latency, and massive resource consumption. However, these "Proof-of-X" algorithms serve as a fundamental theoretical inspiration.

They were explicitly designed to reach consensus in fully untrusted environments where any participating node could be malicious or anonymous. Their success provides a critical lesson for our SPS architecture: in environments susceptible to targeted cyberattacks and partial compromises, we cannot rely on consensus protocols that implicitly and excessively trust the other participating nodes. If a compromised autonomic manager node were implicitly trusted, it could easily broadcast false states or suppress legitimate IDS alerts, completely neutralizing the system's defensive capabilities. Therefore, the chosen algorithm must fundamentally account for and tolerate malicious actors.

2.3.2 Crash Fault Tolerance

Crash Fault Tolerance (CFT) algorithms are built for systems where computers might suddenly crash, lose power, or drop off the network. The core idea behind all CFT algorithms is a strong assumption of honesty. They expect that while a node might fail silently, it will never act maliciously or lie. As long as a majority of the nodes are still running and following the rules, the system can stay consistent. The problem is that this built-in trust makes CFT algorithms incredibly fragile in untrusted environments. If an attacker takes over a node and starts feeding the network bad data on purpose, the entire system can easily be subverted.

- **Paxos:** Introduced by Leslie Lamport, Paxos achieves consensus through a strictly mathematical multi-phase process: *Prepare* (a proposer selects a sequence number and requests promises from acceptors), *Accept* (upon

receiving a majority of promises, the proposer broadcasts the value), and *Learn* (the agreed value is propagated). Its technical complexity arises from managing concurrent proposals and overlapping majorities without a stable leader.

- **Raft:** Designed specifically as a more comprehensible alternative to Paxos, Raft achieves CFT by decomposing the consensus problem into three distinct mechanisms: leader election, log replication, and safety. A strongly elected leader continuously sends *AppendEntries* heartbeats to followers. If a follower's randomized timeout expires without receiving a heartbeat, it transitions to a candidate state to trigger a new election.

2.3.3 Leader-Based Algorithms

Leader-based algorithms try to make things simpler and much faster by putting one single node in charge. This elected coordinator, usually called the leader or primary, takes on the responsibility of organizing transactions, making the final decisions, and telling all the other follower nodes what the new state is. Because everything goes through one point, these systems can process data very efficiently. However, this design creates an obvious weak spot. If the leader crashes, the whole system stops until the followers can run an election and agree on a replacement. Just like general CFT algorithms, traditional leader-based approaches also assume that all nodes are honest, which means they are generally not equipped to handle nodes that try to sabotage the network. Here are reported some common Leader-based consensus algorithms

- **Viewstamped Replication:** A state replication protocol that relies on a primary node to sequence requests within a specific configuration period called a "view". The primary assigns monotonic sequence numbers and multicasts operations to backups. If the primary fails, the replicas initiate a view change protocol, electing a new primary via a deterministic round-robin schedule (based on the view number modulo the number of nodes) and synchronizing log entries before resuming.

- **Bully Algorithm:** A classic leader election algorithm where participating nodes are assigned unique, strictly hierarchical identifiers. When a node detects a failure, it sends an *ELECTION* message to all nodes with higher IDs. If no node replies, it assumes leadership and broadcasts a *COORDINATOR* message. If a higher-ID node replies, the lower-ID node yields. Crucially, if a crashed node with a high ID recovers, it immediately preempts the current leader.

2.3.4 Byzantine Fault Tolerance

Byzantine Fault Tolerance (BFT) algorithms solve the biggest problem with CFT by preparing for the worst. They are designed to keep the network running correctly even if a specific portion of the nodes fails completely, goes offline, or is actively hijacked by an attacker trying to spread conflicting information. For a network of $3f + 1$ nodes, up to f nodes can turn fully malicious without breaking the system. It gives us the strong security we need to survive compromised nodes, while still being fast and efficient enough for a permissioned environment without needing the heavy and wasteful mechanics of "Proof-of-X".

To guarantee that malicious nodes cannot trick the network into creating a fork, BFT algorithms rely on a strict multi-phase voting process. While different protocols use slightly different names, the core structure is the same. First is the **Propose** (or *Pre-prepare*) phase, where a designated leader broadcasts a new block. Then comes the **Prevote** (or *Prepare*) phase, where all other nodes verify the block and broadcast a vote to confirm they received a valid proposal. Finally, the **Precommit** (or *Commit*) phase begins once a node sees that a supermajority of the network has prevoted. In this final step, nodes broadcast their final approval. Only when a supermajority of precommit votes is collected does the block get permanently written to the ledger. This double-checking mechanism ensures that no honest node commits a block unless it is absolutely certain the majority of the network is doing the same.

- **QBFT (Quorum Byzantine Fault Tolerance):** A consensus algorithm designed specifically for permissioned, Ethereum-based networks like Quorum. It operates through this deterministic, three-step voting process (*Pre-prepare*, *Prepare*, *Commit*) driven by a proposer selected via a round-robin schedule. A block requires $2f + 1$ cryptographic validation signatures from a predefined pool of $3f + 1$ validators to be committed, yielding immediate transaction finality without the possibility of forks.

- **Tendermint/CometBFT:** The consensus engine underlying CometBFT provides highly efficient BFT state machine replication. It enforces a strict separation between the application logic and the consensus layer via the Application Blockchain Interface (ABCI). Tendermint utilizes a continuous gossiping protocol and requires a two-thirds supermajority of voting power at both the *prevote* and *precommit* stages. Unlike traditional periodic block production, its event-driven model generates blocks only when transactions are pending.

2.4 Autonomic Manager Security's Current State

As previously discussed, the question of "Who watches the watchers?" remains a central dilemma in the field of autonomic computing. If an adversary successfully compromises the autonomic manager, they can blind the system to ongoing attacks or even weaponize its response mechanisms against the underlying infrastructure. Over the years, various architectural and technological paradigms have been proposed to secure the manager, each presenting distinct advantages and critical limitations.

2.4.1 Traditional Hardening and Centralized Security

The most conventional approach to protecting the manager relies on defense-in-depth strategies deployed within a centralized architecture. This typically involves isolating the autonomic manager within a highly secure environment, fortified by strict network firewalls, stringent access controls, and dedicated monitoring infrastructure. While straightforward to implement, this approach inherently relies on securing a single point of failure rather than providing intrinsic architectural resilience. If an adversary exploits a zero-day vulnerability in the manager's dependencies or breaches the surrounding network perimeter, the system's entire defensive capability is instantly compromised. Consequently, while system hardening remains a mandatory prerequisite for any component interfacing with untrusted actors, it is insufficient as a standalone security paradigm.

2.4.2 Trusted Execution Environments (TEEs)

To fundamentally reduce the attack surface, researchers have explored hardware-assisted isolation using Trusted Execution Environments (TEEs), such as Intel Software Guard Extensions (SGX) that imposed itself as the current state-of-the-art in the field. In this model, the autonomic manager's critical logic and sensitive data are executed within a secure hardware enclave. Even in the event of a complete compromise of the host operating system or hypervisor, the adversary theoretically cannot read or tamper with the manager's execution. This is achieved through transparent memory encryption and strict privilege separation, the TEE operates at a highly privileged hardware level, ensuring that untrusted software running in higher-level protection rings cannot access the enclave's memory space.

Despite these strong theoretical guarantees, TEEs exhibit critical practical limitations. Operating on shared physical hardware exposes the enclave to a broad spectrum of microarchitectural side-channel attacks. High-profile vulnerabilities such as Spectre and Meltdown exploit speculative execution and CPU caching mechanisms to leak enclave secrets. Software patching mitigations rarely eliminate

the entire attack surface, as evidenced by the subsequent emergence of sophisticated attacks like Microarchitectural Data Sampling (MDS, better known as ZombieLoad). Finally, relying on TEEs introduces severe vendor lock-in and fails to resolve the single-point-of-failure problem, an adversary can simply disconnect or crash the physical machine hosting the enclave, resulting in a complete Denial of Service (DoS).

2.4.3 Distributed Ledger Technologies

As established in Section 2.2, integrating permissioned blockchain technology directly into the autonomic manager provides a robust architectural solution. By replicating the manager's state across multiple independent nodes and encoding the response logic within deterministic smart contracts, the system fundamentally eliminates the centralized weak spot while simultaneously maintaining an immutable, cryptographically verifiable audit trail of all security events.

Crucially, implementing this distributed architecture requires a rigorous protocol to manage node coordination in an environment susceptible to malicious actors. As analyzed in Section 2.3, Byzantine Fault Tolerance (BFT) algorithms provide the exact mathematical guarantees needed for this task. By requiring a supermajority agreement for every state transition, the distributed manager can securely tolerate partial network compromise without halting operations or accepting tampered data. This synergy of blockchain architecture and BFT consensus can resolve the foundational dilemma of autonomic computing with a structural solution, redefining the manager from a vulnerable isolated entity into a resilient, decentralized and physically separate network.

Chapter 3

Self-Protecting Systems Architecture

This chapter details the architectural core of the proposed blockchain-based Self-Protecting System (SPS), focusing on how to address the centralized autonomic manager's problem explored in Section 2.4. The model presented herein is grounded in the foundational decentralized architecture introduced in [2] and subsequently refined through the requirement-driven analysis by our research [10]. The primary objective is to formalize the system decomposition, state model, end-to-end protection workflow, threat assumptions, and resilience properties. These components justify the integration of a permissioned, Byzantine Fault Tolerant (BFT) distributed ledger as the secure decision substrate for the autonomic manager.

3.1 Design Principles and Rationale

Traditional SPSs are typically designed around a centralized MAPE-K feedback loop [3]. While functionally effective, this centralized model introduces a fundamental security asymmetry: the managed system can be replicated, segmented, and protected, whereas the autonomic manager itself remains a high-value, centralized target. If an adversary successfully compromises the manager, they can suppress valid security alerts, trigger unauthorized response actions, or stall mitigation entirely. This effectively neutralizes the self-protection mechanism and grants the attacker complete control over the production environment [6, 7].

To address this structural vulnerability, the architecture proposed in this thesis distributes the decision authority of the autonomic manager across replicated, isolated nodes coordinated through a permissioned blockchain using BFT consensus [2]. The core design choice is not simply the adoption of blockchain as a modern storage medium, but its use as a replicated state machine in which:

- **Consensus-driven state updates**: Monitored variables representing the managed system state are updated only after achieving quorum-based agreement among independent observers;

- **Deterministic transitions**: Response decisions are modeled as deterministic functions of the globally agreed state, guaranteeing that honest nodes reach identical conclusions;
- **Auditable actuation**: Actuation instructions are pushed from a tamper-evident, globally replicated log, ensuring source accountability and post-incident forensic auditability;

By encoding the autonomic manager within a Byzantine Fault Tolerant ledger, trust is shifted from a single, hardening-dependent process to a formal fault model. The central security question shifts to determining the number of components that can fail or be corrupted before the safety and liveness of the control loop are violated, exposing the managed system to threats.

At the architectural level, this model is deliberately technology-agnostic. It does not dictate a specific Intrusion Detection System (IDS) engine, response planner, or blockchain implementation. Instead, it defines a strict security contract among the components: authenticated observations, deterministic state transitions, replicated BFT consensus, and auditable, blind actuation. A crucial aspect of this design is the requirement for *immediate finality* in the underlying consensus protocol. Unlike public permissionless blockchains that rely on probabilistic finality (e.g., waiting for multiple block confirmations), a software control loop cannot act on decisions that might be subsequently rolled back. A block commit must represent a final, immutable transition to avoid executing conflicting or unauthorized actions.

3.2 Architectural Decomposition and Trust Boundaries

To ensure structural resilience, the architecture decomposes the system into distinct functional planes, separated by explicit trust boundaries and network isolation. The system maintains the classic autonomic split between the *managed system* (the production environment) and the *manager system* (the security controller) [3, 2], but introduces strict boundaries to limit the lateral propagation of attacks.

3.2.1 Managed System

The managed system represents the production workload to be protected (e.g., containerized application services, microservices, or database nodes). From a security perspective, this domain is assumed to be untrusted, as it is directly exposed to external users and potential attackers. The behavior and operational

health of the managed system are captured through a set of monitored variables:

$$\mathbf{V} = \{v_1, v_2, \ldots, v_k\}.$$

These variables represent resource-level, network-level, or application-level signals (such as CPU saturation, database query anomalies, authentication failure rates, or protocol misuse indicators). At any time instant t, the observable state of the managed system is represented by the vector:

$$s_t = \langle v_1(t), v_2(t), \ldots, v_k(t) \rangle.$$

Security-relevant events (e.g., a Denial of Service attack or a SQL injection attempt) modify one or more coordinates of s_t. The architecture does not couple itself to any specific detection paradigm, allowing the use of signature-based, anomaly-based, or hybrid detection systems to monitor these variables.

3.2.2 Manager System

The manager system is the Control plane in charge of processing state observations and executing responses. It is decomposed into three logical planes:

1. **Detection Plane**: Composed of multiple independent IDS instances that observe overlapping portions of the managed system's state variables $\mathbf{V}$.
2. **Consensus-and-Decision Plane**: A permissioned blockchain network that maintains the replicated decision state and executes the response selection logic.
3. **Control Plane**: A set of response executors (actuators) that receive committed mitigation commands and enforce them on the managed system.

Each active component in the Detection and Control planes is paired with a blockchain-facing node. In our design, we distinguish between *validating nodes* (which participate in the BFT consensus protocol) and *member or light nodes* (which submit transactions or listen to events without validating blocks). This separation decouples functional roles from consensus overhead, allowing the validator set to remain small and efficient while the number of sensors and actuators scales.

3.2.3 Isolation and Containerization Requirements

The security of the manager system relies heavily on containerization and network isolation [2]. Each IDS, actuator, and blockchain node must run in its own isolated container, deployed across physical or virtual hosts. In our prototype, this environment is emulated using the Kathará framework [11] on top of Docker.

Under this model, the container boundary serves as the primary security perimeter. If an attacker compromises an application service in the managed system, they cannot pivot into the manager system's containers unless an explicit, authenticated communication path exists. The attack surface of the manager system is restricted to three channels:

- The telemetry and input channels of the IDSs,
- The transaction submission APIs exposed by the blockchain nodes,
- The control paths through which actuators apply countermeasures.

3.2.4 Actuator Blindness and Unidirectional Control

To prevent attackers from manipulating responses, actuators are modeled as *blind* [2]. Actuators do not query the managed system directly to inspect its state, nor do they accept instructions from it. Instead, they operate exclusively as passive consumers of the committed decisions stored in the blockchain.

If an actuator were allowed to pull state variables directly from a compromised application container to confirm an attack, the attacker could exploit this interactive channel to inject malicious payloads or trigger unauthorized actions. By enforcing actuator blindness, the control loop establishes a strict unidirectional data flow: IDSs push observations to the consensus plane, the blockchain validates and commits the state transition, and the resulting response is pushed to the actuators. This design pattern ensures that an actuator only executes commands that have passed through BFT consensus.

3.3 System's State Formalization

To prevent ambiguity and support formal verification, the control pipeline of the decentralized manager is formalized as a deterministic state machine.

3.3.1 Observation Sets and Redundancy

Let $\mathcal{D} = \{d_1, d_2, \ldots, d_m\}$ be the set of deployed IDSs. Each IDS d_i is configured to monitor a specific subset of state variables, denoted by the observation scope $O_i \subseteq \mathbf{V}$. For any monitored variable $v_j \in \mathbf{V}$, we define its observer set $\mathcal{D}_{v_j}$ as the subset of IDSs that monitor it:

$$\mathcal{D}_{v_j} = \{d_i \in \mathcal{D} \mid v_j \in O_i\}.$$

To ensure semantic robustness, the architecture requires $|\mathcal{D}_{v_j}| \geq 3$ for all critical variables, utilizing heterogeneous detection technologies (e.g., combining Snort,

Suricata, and Zeek). Using diverse detection engines ensures that a zero-day exploit or signature bypass targeting one specific IDS implementation will not compromise the consolidated observation, as the remaining heterogeneous observers can still report the event accurately.

3.3.2 Proposal and Consolidation Semantics

When an IDS d_i detects a change in a variable v_j, it submits a state proposal transaction to the blockchain:

$$p = \langle id_{ids}, v_j, \hat{x}, t \rangle$$

where id_{ids} is the unique identifier of the submitting IDS (d_i), v_j is the target variable, $\hat{x}$ is the proposed value or classification, and t is the detection timestamp.

Inside the smart contract or the application state machine, incoming proposals are collected within a sliding aggregation window. To filter out noise or a compromised sensor's false reports, the state is updated only when a quorum predicate is satisfied. In its simplest majority form, the consolidation predicate is defined as:

$$count_{v_j}(\hat{x}) \geq q_j, \quad q_j = \left\lfloor \frac{|\mathcal{D}_{v_j}|}{2} \right\rfloor + 1.$$

In stricter Byzantine configurations with $3f_d + 1$ observers, the quorum threshold can be adjusted to $2f_d + 1$ to tolerate up to f_d Byzantine (malicious or failed) IDSs.

3.3.3 Replicated Decision State

The global state of the decentralized manager is formalized as:

$$\Sigma_t = \langle S_t, R_t, I_t \rangle$$

where:

- S_t is the consolidated monitored state (represented as a parameter-status map),
- R_t is the response relation (represented as a state-action map mapping consolidated states to countermeasures),
- I_t is the internal control metadata (such as active response flags, execution status, and cooldown timers).

Every committed transaction deterministically transforms the manager state: $\Sigma_t \xrightarrow{T_i} \Sigma_{t+1}$. To represent this state in a concrete implementation (such as the

prototypes analyzed in Chapters 4 and 5), the consolidated monitored state S_t is modeled as a bit-vector (a sequence of boolean flags) of length N, where N represents the number of distinct attacks or anomalies detectable by the system. The response relation R_t is implemented as a map associating each state vector with a specific mitigation action. For example, in our prototype, R_t maps state vectors to command strings or script identifiers that the actuators are authorized to execute.

3.3.4 Determinism and Auditability

Deterministic execution is a fundamental requirement for the manager state machine. Given the same initial state Σ_0 and an identical sequence of transactions $T_1, T_2, \ldots, T_n$, all non-faulty validator nodes must compute the exact same sequence of states $\Sigma_1, \ldots, \Sigma_n$ and produce the exact same outputs. This provides two key security benefits:

- **Consistency**: It prevents split-brain scenarios, ensuring that all replicas agree on which response action to trigger,
- **Forensics**: The entire history of committed transitions is recorded in an immutable ledger, allowing post-incident reconstruction and verification of security policies.

To preserve determinism across the system, both the state transition function (executed by the consensus nodes) and the alert generation process (performed by the detection components) must adhere to these constraints. For instance, any runtime logic or actuator selection mechanism must avoid relying on non-deterministic variables (such as local clocks or system randomness) so that all replicas compute identical state updates.

3.3.5 State-Action Map Limitations and Temporal Context

A direct state-action map assumes a Markov-like property, meaning that the current consolidated state S_t contains all the information necessary to select a response. While this simplifies the design and guarantees deterministic replay, it limits the manager's ability to detect multi-stage or low-and-slow attacks that unfold over time.

To overcome this limitation, the state representation must incorporate historical context directly into the state variables. This can be achieved by storing memory variables (such as event counters, sliding-window registers, and cooldown flags) within the internal metadata I_t. Rather than relying on external logs, the state machine itself tracks temporal patterns, ensuring that the inputs to the response logic remain self-contained and deterministic.

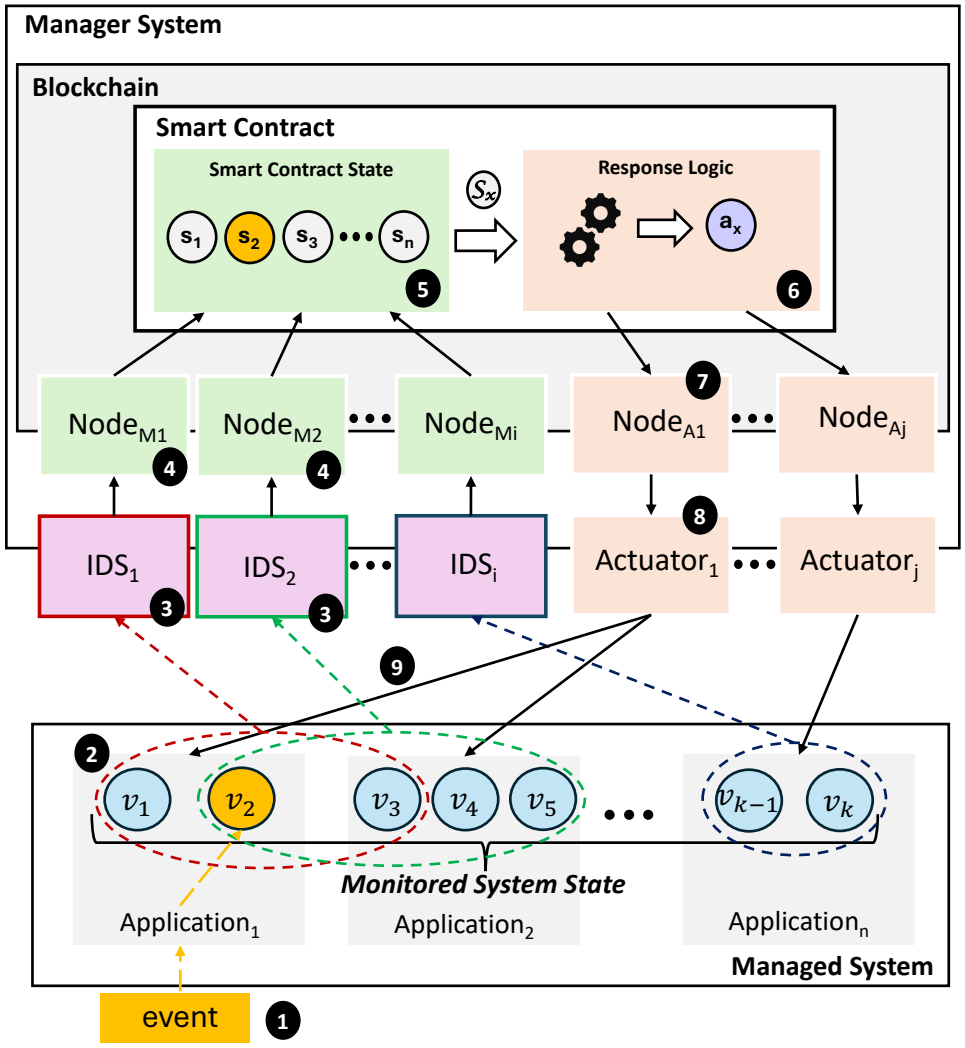

Figure 3.1: A formalized representation of the SPS in all it's components.

3.4 End-to-End Protection Workflow

The proposed architecture establishes a closed-loop control pipeline that processes security events from detection to mitigation. The workflow consists of four sequential, causally ordered stages:

3.4.1 Managed Monitoring and Attack Detection

When a malicious or anomalous event occurs in the managed system, it alters one or more monitored variables. The independent IDSs observing these variables detect the anomaly and evaluate the event type. Because different IDSs utilize different detection engines (e.g., signature-based vs. protocol analysis) and run on distinct containers, they may flag the attack at slightly different times, introducing a temporal skew. The architecture accommodates this asynchrony by aggregating alerts incrementally on the blockchain's smart contract.

3.4.2 Alert and Transaction Submission

The detecting IDS sends an alert to an associated Member Node that wraps the observation into a proposal, signs it cryptographically, and submits it as a transaction to its paired blockchain node. The entire permissioned blockchain network receives the transaction and verifies the signature against the membership service. Because the blockchain nodes are isolated from the untrusted managed system, an external attacker who has compromised an application container cannot easily inject forged transactions or manipulate the consensus membership without compromising multiple IDSs. This process is discussed in more detail in Section 3.6.6.

3.4.3 Consensus-Guided State Transition

The validating nodes receive the verified transactions, order them using the BFT consensus protocol, and package them into blocks. Once a block is committed, the transitions are executed by the smart contract (or the ABCI application). If the aggregated proposals satisfy the voting predicate, the consolidated state S_t is updated, and the response logic determines the appropriate countermeasure a_t. This BFT-guided commitment guarantees that the response decision is final and agreed upon by a qualified majority of the network.

3.4.4 Response Actuation and Feedback Loop

Once a response a_t is committed, the blockchain node notifies the selected actuator using a push-based subscription channel (e.g., Ethereum event logs or CometBFT Websocket events). The blind actuator receives the instruction and executes the mitigation action (e.g., blocking an IP address, updating firewall rules, or restarting a container) via a secure channel such as SSH.

The enforcement of the countermeasure alters the managed system's state, returning the monitored variables $\mathbf{V}$ to their normal, secure range. The IDSs observe this return to a safe state and submit proposals to clear the active threat state. The state machine then updates, clears the active response flag, and terminates the response execution, closing the feedback loop.

3.5 Correctness and Actuation Properties

To evaluate the reliability of the Control plane, we define formal correctness properties that must be preserved by the architecture.

3.5.1 Safety Invariants

Safety in the Control plane requires that the manager never triggers unauthorized or conflicting actions. We express this through two formal invariants:

1. **No Unauthorized Actuation**: Let A be the set of mitigation actions, and let $E_{exec}(a)$ denote the event that action $a \in A$ is executed by an actuator. Let $\Sigma_{commit}(a)$ denote the event that the replicated state machine commits a transition emitting the response action a. The safety invariant requires:

   $$E_{exec}(a) \implies \Sigma_{commit}(a)$$

   This is guaranteed by actuator blindness and push-based communication channels.

2. **Agreement (No Conflicting Decisions)**: For any block height h, and any two non-faulty consensus nodes i and j, their committed states must be identical:

   $$\Sigma_h^i = \Sigma_h^j$$

   This is guaranteed at the blockchain layer by BFT consensus protocols, but must be properly addressed in the smart contract implementation for IDS-level voting.

The combination of these two invariants is critical. Reaching consensus without actuator discipline (blindness) could still allow unauthorized local execution, while actuator discipline without consensus could result in divergent control histories across different nodes.

3.5.2 Liveness Guarantees

Liveness ensures that the system reacts to threats in a timely manner. Formally, if a security breach changes the monitored variables v_j at time t such that the response logic requires action a, then a non-faulty actuator must execute a at some time $t' > t$, assuming that the number of Byzantine failures does not exceed the consensus protocol's threshold.

Liveness is practically constrained by validator availability, transaction submission latency, and block production times. The block production model of the blockchain backend is a relevant parameter when choosing the technology to implement the system and will be addressed in Chapter.

3.6 Failure, Recovery and Security

A resilient SPS must maintain predictable behavior not only during active cyber-attacks but also during system faults, partitions, and recovery events.

3.6.1 Crash and Restart of Non-Validator Components

If an IDS container crashes, the system's detection coverage temporarily decreases, but the consensus and decision plane continues to operate. Upon restart, the IDS re-authenticates and resumes submitting proposals without requiring a system-wide reconfiguration thanks to the self-healing properties of containerized applications.

If an actuator container crashes and restarts, it must not replay stale mitigation actions. To prevent this, the actuator queries the internal metadata I_t on the blockchain. If the metadata indicates that the action associated with the current state has already been successfully executed, the actuator suppresses execution, preventing duplicate actuation cycles that could disrupt the managed system.

3.6.2 Validator Failure and Quorum Loss

Let N_v be the total number of validator nodes in the permissioned network. In a standard BFT consensus protocol, the quorum size Q_v required to commit a block is:

$$Q_v = \left\lfloor \frac{2N_v}{3} \right\rfloor + 1.$$

The system can tolerate up to $f_v = \lfloor \frac{N_v - 1}{3} \rfloor$ Byzantine (malicious or crashed) validators.

If the number of failed validators remains within this threshold ($f \leq f_v$), the network continues to commit transactions and guarantees safety and liveness. If the number of failures exceeds f_v, the consensus protocol stalls (safety-preserving stall). While the manager ceases to process new state updates, it preserves its last committed state, avoiding split-brain or divergent state transitions.

3.6.3 Network Partitions

In the event of a network partition, the CAP (Consistency, Availability, Partition tolerance) theorem applies:

According to the CAP theorem, it is impossible for a distributed data store to simultaneously provide more than two out of the following three guarantees: Consistency (C), Availability (A), and Partition Tolerance (P). Formally, given a network partition (P), the system cannot maintain both consistency and availability,

which can be expressed as:

$$P \implies \neg(C \wedge A)$$

Consequently, under a network partition, a system must make a trade-off, leading to either a CP (Consistency/Partition Tolerance) or an AP (Availability/Partition Tolerance) design strategy.

The BFT-based manager prioritizes consistency (safety) over availability (liveness). Only the partition containing a supermajority of validators (Q_v) can continue committing state transitions. Minority partitions are unable to reach consensus and stall. While this temporarily reduces responsiveness in isolated segments, it prevents the creation of divergent control histories, preserving the integrity of the global state.

3.6.4 Threat Model

The threat model assumes an active adversary with network access to the managed service, whose objective is to subvert the autonomic manager. A successful attack on the manager is defined by two failure modes:

1. **Omission**: An attack occurs and is detected, but the manager fails to execute the required response,
2. **Illegitimate Actuation**: An actuator executes a countermeasure that was not authorized by the consolidated manager state.

The threat model is defined under the following assumptions:

- The container isolation primitive is robust, preventing container escapes,
- Network segments connecting the manager (IDSs and Blockchain) containers are isolated from public traffic,
- The only direct, untrusted inputs entering the manager are the exposed variables monitored by the IDSs (e.g., the network traffic),
- The smart contract or ABCI state machine code is correctly and securely implemented, and
- Node identities are verified and governed by a permissioned membership model.

3.6.5 Adversarial Strategies

An attacker attempting to subvert the manager can pursue three main strategies:

1. **Input Poisoning**: The attacker attempts to compromise multiple IDSs to inject false observations, forcing the manager into an incorrect state.

2. **Consensus Subversion**: The attacker attempts to compromise more than 1/3 of the validator nodes to take control of block commitment and rewrite history.

3. **Actuator Corruption**: The attacker attempts to compromise the actuators directly to block mitigations or trigger unauthorized actions.

The architecture raises the cost of each strategy:

- Input poisoning is mitigated by the multi-IDS voting logic and the use of heterogeneous detection engines that prevent technology-specific faults,
- Consensus subversion is prevented by BFT consensus, which requires compromising a supermajority of validators, and
- Actuator corruption is mitigated by actuator blindness, which prevents direct communication from compromised application containers.

3.6.6 Voting and Validation Differences

This section formalizes the security guarantees of the architecture, showing how the remaining attack paths are restricted and highly resource-intensive for an adversary. Let N_v be the number of validators in the consensus layer. The system can tolerate up to f_v Byzantine validators, where:

$$f_v = \left\lfloor \frac{N_v - 1}{3} \right\rfloor.$$

Let N_d be the number of independent IDSs monitoring a state variable v_j. If majority voting is applied, the observation layer can tolerate up to f_d compromised IDSs, where:

$$f_d = \left\lfloor \frac{N_d - 1}{2} \right\rfloor.$$

To successfully subvert the manager, an attacker must compromise either a majority of the IDSs observing a target variable ($> f_d$) or a supermajority of the consensus validators ($> f_v$). This two-layer threshold design prevents the single point of failure inherent in centralized autonomic managers.

Redundancy without diversity is vulnerable to common-mode failures. If all IDSs run the exact same software, a single vulnerability could allow an attacker to compromise the entire set simultaneously. The architecture therefore recommends using heterogeneous IDS engines (e.g., Snort, Suricata, and Zeek). This increases

the attacker's costs by requiring them to develop and deploy multiple distinct exploits, making the possibility of compromising the autonomic manager highly improbable since the architecture can increase the number of diverse technologies at will.

3.6.7 Residual Risk

The architecture does not protect against host-level compromises or vulnerabilities in the underlying container runtime. If an attacker gains root access to the physical host, they can bypass container boundaries and compromise the nodes. Additionally, bugs in the smart contract or ABCI code remain a critical vulnerability, as a replicated state machine will faithfully execute and replicate flawed logic. In any computer system, expanding the trusted computing base is a choice that must be treated with caution, therefore, secure coding and formal verification of the response transition function remain essential.

Chapter 4

Requirement Alignment: Quorum vs. CometBFT

This chapter evaluates how Quorum and CometBFT align with the requirements derived in the preceding analysis of our SPS-architecture. The focus is on understanding the practical match between those requirements and the design assumptions of the two platforms. In this sense, *alignment* means that the distributed platform's design choices, operational model, and implementative costs tightly fit the requirements of a cyber-resilient SPS's autonomic manager, without adding functionality that is either irrelevant or actively counterproductive in our application domain.

The SPS manager is essentially a replicated, deterministic coordinator that receives sparse alerts, aggregates them, and triggers countermeasures. This is a narrow use case compared to the general-purpose decentralized and firmly untrusted application ecosystems that motivated the existence of the blockchain as we know it. Therefore, the most relevant dimension of comparison is whether each platform supports the required properties *as first-class, low-overhead features*, and whether it avoids non-needed functionality that increases the trusted computing base or injects avoidable delay. Table 4.1 summarizes the high-level mapping, the remainder of this section justifies each cell in that mapping and discusses the trade-offs that arise from different design philosophies.

Table 4.1: Mapping of Quorum and CometBFT features to SPS blockchain requirements.

Property	Quorum (IBFT/QBFT)	CometBFT (ABCI)
Required		
BFT consensus with finality	Native	Native
Permissioned static membership	Native	Programmable
Transaction signing	Native	Native
Event-driven block production	Limited	Native
Desirable		
In-memory replicated state	Partial	Flexible
Low and predictable latency	Fair	Excellent
Graceful scaling with nodes	Limited	Programmable
Configurable audit immutability	Limited	Supported
Push-based notification	Native	Programmable
Not needed		
Fixed-interval block production	Present	Absent
Gas / fee mechanisms	Present	Absent
Token incentives	Inherited	Absent
PoW / PoS	Inherited	Absent
Dynamic peer discovery	Present	Absent
General-purpose VM	Present	Absent

4.1 Alignment lens and scope

The comparison is requirement-driven rather than feature-driven. It does not reward a platform for offering additional services, such as token economy, unless those services directly improve the SPS manager's correctness, latency, or operational resilience. This is a deliberate lens: if a feature is not needed by the SPS manager, then its *best* contribution is to be absent, because any additional subsystem increases configuration complexity or waste computational power also risking to create new failure modes or attack vectors that must be secured.

We also separate *native* alignment (a property provided by the platform itself) from *programmable* alignment (a property that can be implemented in the application layer). In SPS, programmable alignment can be acceptable, but only when the implementation effort is justified by measurable benefits such as lower latency, reduced attack surface, or greater determinism. This distinction is crucial when comparing a full-stack platform like Quorum with a consensus engine like CometBFT that intentionally pushes application logic to the developer, it is notable to remark that higher deegrees of freedom come with a responsibility shift on the

developer implementing the system.

4.2 SPS Required Properties

4.2.1 BFT consensus with deterministic finality

Quorum satisfies this requirement through IBFT and QBFT. Both protocols require a super-majority of validators (at least 2/3) to sign each block, blocks are final and do not fork, and the system cannot progress if more than one third of validators are offline or Byzantine [12, 13]. This behavior fits the SPS manager's safety needs because it provides a single authoritative state even under partial compromise.

CometBFT was explicitly designed to provide Byzantine fault-tolerant state machine replication with deterministic finality [14, 15]. It offers the same safety guarantees as IBFT/QBFT in terms of tolerating up to f Byzantine validators in a set of $3f + 1$. The key difference is where the application logic lives: Quorum merges consensus and application in an Ethereum Virtual Machine (EVM), while CometBFT keeps consensus separate and exposes application logic through ABCI (Application BlockChain Interface) and keep the consensus a black box to the application. For SPS, both satisfy the main constraint of BFT, but CometBFT achieves it with less embedded application machinery, reducing drastically the overhead introduced to compute a whole EVM virtualization.

4.2.2 Permissioned static membership

The SPS trust boundary is fixed, new membership require a reconfiguration of the system, so validator membership should be permissioned and only change under explicit governance. Quorum supports this natively through validator set management in IBFT and QBFT. Validators are approved accounts, and membership can be changed through voting, either by JSON-RPC methods or via a governance smart contract [12]. This closely matches the requirement, but the contract-based approach introduces additional on-chain logic that must itself be secured.

CometBFT does not include a built-in permissioning layer. Membership is defined in the genesis configuration and can be updated by the application through ABCI. This means static membership can be enforced as a policy in the application logic, which is adequate for SPS but requires additional engineering. The trade-off is clear: Quorum delivers the feature out of the box, while CometBFT makes the designer explicitly encode the governance rules that fit the SPS deployment.

4.2.3 Transaction signing and accountability

Transaction signing is a native property of the Ethereum lineage on which Quorum is built, every transaction carries a signature and is authenticated by the consensus layer. For the SPS manager, this provides accountability for IDS nodes and actuators, enabling non-repudiation and forensic auditing.

In CometBFT, transaction validation is explicitly delegated to the application through ABCI. The standard pipeline calls `CheckTx` for mempool admission and later `ProcessProposal` or `FinalizeBlock` for deterministic execution, and these hooks are the natural place to verify signatures and enforce authorization policies [16]. This yields the same functional property as in Quorum, but the responsibility for signature schemes and key management is pushed to the application.

4.2.4 Event-driven block production

Alert traffic in SPS is sparse and bursty due to the nature of cyber-attacks, which makes fixed-interval block production a poor fit. Quorum's QBFT configuration uses a block period timer (`blockperiodseconds`) that triggers proposal attempts at a minimum fixed interval [12]. This keeps consensus active even under idle conditions and introduces a baseline delay between detection and commitment that is independent of application complexity, this is acceptable but not ideal.

CometBFT allows disabling empty block creation through configuration, effectively enabling blocks only when transactions exist [17]. This matches the SPS requirement for event-driven progression: the system can remain quiescent without consuming consensus cycles, and when an alert arrives it can be committed without waiting for the next periodic block. In requirement alignment terms, CometBFT is closer to the desired behavior and can be tuned to minimize idle overhead.

4.3 Desirable properties: efficiency and responsiveness

4.3.1 Execution model and deterministic state transitions

Quorum inherits the Ethereum execution model. Transactions are RLP-encoded, executed in the EVM, and charged gas at instruction granularity [9]. Even in permissioned deployments where fees are irrelevant, the EVM still enforces gas accounting and executes bytecode in a general-purpose virtual machine. The SPS response logic is a small deterministic automaton, it uses only a narrow subset of

EVM's instrumentation while paying the full runtime cost and complexity of a general-purpose environment.

CometBFT actuates the execution through ABCI, allowing the SPS state machine to be implemented in a native language such as Go or Rust. This eliminates bytecode interpretation and gas accounting overhead, and it makes the runtime cost of a state transition proportional to its intrinsic logic rather than to virtual machine scaffolding. ABCI also imposes strict determinism requirements on several handlers (for example, `ProcessProposal` must be deterministic) [16]. This constraint is compatible with SPS, which already requires deterministic logic, but it places additional discipline on the implementation to avoid non-deterministic libraries, wall-clock time, or concurrency races that would diverge across replicas.

4.3.2 State storage and in-memory replication

The SPS state is small and stable, which makes in-memory replication attractive. Quorum, as an Ethereum client, persists its state in a database optimized for large smart-contract state and historical storage. This persistence is beneficial for general-purpose applications, but in SPS it adds I/O overhead and enlarges the system's storage footprint without clear benefit, especially when the response logic only needs the current state and short-term history.

CometBFT separates consensus from application state, so the application is free to choose storage strategies. The core node still persists its consensus metadata and block store, but the application can keep its own replicated state in memory and persist only what is operationally required. This flexibility aligns with the desirable requirement for in-memory state and opens the possibility of deployement on resource-constrained systems, allowing to estimate in advance the required hardware resources.

4.3.3 Latency predictability at low load

SPS effectiveness depends on short and predictable response time. In Quorum, two mechanisms contribute to baseline latency: the fixed block period and the EVM execution overhead. Even when only a single alert transaction is submitted, the transaction must wait for the next block proposal window, consensus then proceeds, and the state transition executes in the EVM with gas metering.

In CometBFT, latency is dominated by the consensus round and the application execution time. When empty blocks are disabled, the first transaction can be proposed immediately without waiting for a periodic timer, also CometBFT offer a wide range of time constraints that approriately choose can offer a lower and more stable response time guaranteeing the consensus within an estimated time

window. For a small validator set, the BFT round completes quickly, and the native application logic executes with minimal overhead.

4.3.4 Burst handling and Timeouts customization

SPS workloads are not only sparse but also bursty: coordinated attacks can trigger multiple IDS alerts in a short window, and a robust system must absorb these bursts without collapsing into unbounded latency. In Quorum, the fixed blockperiod provides a stable clock to the system, but also provide a bound on how quickly the system can process the ledger's data. When alerts arrive faster than blocks can be proposed, transactions stacks in the pool and are executed in subsequent blocks, increasing delay.

CometBFT exposes more opportunities for customization and filtering at the application level. Because `CheckTx` is invoked before a transaction enters the mempool, an SPS application can reject duplicates, apply rate limits, or enforce priority rules based on IDS provenance and severity [16]. This capability does not eliminate the fundamental consensus bottleneck, but it allows the designer to push the protocol's internal timeouts at the minimum required by the implementation and the hardware, to process without waiting extra-time. Such application-level control is aligned with the SPS requirement for bounded latency under attack conditions, even though it requires explicit policy design.

4.3.5 On The Validator Graceful Scalability

BFT consensus protocols exhibit $O(n^2)$ communication in the voting phase, which limits scalability as the validator set grows [8]. Quorum's IBFT/QBFT adheres to this pattern: validators exchange prepare and commit messages with all others, and the protocol stalls if more than one third of validators are offline. In SPS a low number of validator is acceptable but in larger production system it could be usefull to gracefully scale to a slightly larger amount of nodes to provide relyability

The scalability problem is part of a broader trade-off among decentralization, security, and performance that appears in many blockchain designs [18]. SPS deployments typically prioritize security and availability over large-scale decentralization, which makes small validator sets acceptable. However, even within this constrained regime, the system should degrade gracefully as additional validators are added for resilience. Quorum's protocol does not change its message complexity as the validator set grows, so communication cost increases quadratically with each added node.

CometBFT uses gossip dissemination to broadcast proposals and transactions, which reduces the leader's broadcast burden and can lower propagation latency compared to direct full-mesh dissemination [14]. However, the final voting phase

still requires super-majority signatures, and the $O(n^2)$ full-mesh communication remains. In requirement terms, neither platform offers a built-in path to graceful scaling with large validator sets but CometBFT does better in terms of propagation and also offer light nodes member configuration that unload weight from validators to an associate single full-node in charge of communicating with its light nodes, effectively resulting in a semi-hierarchy that can in some use-cases reduce scaling introduced overhead.

4.3.6 Configurable audit immutability

Auditability is useful for forensic analysis, but in SPS it is not always necessary to retain complete transaction history indefinitely. Quorum, following Ethereum's model, is optimized for immutable ledger history and full archival state, which is consistent with public-blockchain assumptions but potentially excessive for closed, resource-constrained SPS deployments.

CometBFT's separation of concerns enables a more flexible policy: the application can decide which events must be recorded, which can be summarized, and which can be discarded after a retention window. This does not weaken consensus safety, but it does require explicit design to ensure that the retained data still supports accountability and audit obligations. From an alignment perspective, CometBFT better supports configurable immutability because it does not assume a fixed archival model.

4.3.7 Push-based notifications

SPS actuators benefit from push-based notification, since polling introduces additional latency. Quorum supports event logs and client subscriptions through the Ethereum API, which can be used to trigger actuation when the smart contract emits a state-change event. This capability is mature and widely supported in Ethereum tooling.

In CometBFT, events can be emitted by the application during block finalization and consumed by clients through the node's RPC event subscription mechanism. As expected this requires explicit wiring in the application layer, it gives the SPS designer control over the semantics and granularity of notifications but also require the functionality to be correctly implemented. Both platforms can support push-based actuation, Quorum's maturity can be usefull in this case but CometBFT doesn't lack of this feature.

4.4 Alignment with the SPS control loop

Self-protecting systems are commonly framed as a control loop (e.g., Monitor, Analyze, Plan, Execute as seen before) operating on a shared knowledge base [3, 4]. The blockchain layer, in this context, act as the coordination substrate of that loop.

The SPS manager's internal state is a compact representation of monitored variables and response states, which can be modeled as a finite-state machine [19]. Quorum allows this logic to be expressed in Solidity, however, the EVM representation tends to be more verbose than the conceptual model, and the state machine structure is often obscured by contract bookkeeping, gas accounting, and event emission. CometBFT permits the same state machine to be implemented as a direct data structure with explicit transitions, which makes the state logic more transparent and easier to map to the formal model used during SPS design.

The MAPE-K perspective also highlights how knowledge updates should be triggered. In an SPS, the knowledge base is reflected in the detection and response logic and should be aligned with the organization implementing the system's requirements. Quorum's periodic block production effectively introduces a time-based update cadence even when no new evidence exists, whereas CometBFT can be configured to update the shared state only when new evidence appears. This difference does not change correctness, but it affects how tightly the blockchain layer is coupled to the semantics of monitoring and analysis, and therefore how cleanly the SPS control loop can be interpreted.

The voting rules that protect against compromised IDSs can be placed in different phases of the transaction pipeline. In Quorum, the typical implementation encodes those rules inside the smart contract, so every alert travels through consensus before being filtered. This preserves a complete audit trail, but it also increases the volume of consensus traffic and imposes EVM execution overhead also for inputs that will ultimately be discarded. In CometBFT, the ABCI interface allows preliminary checks in `CheckTx` and proposal-level validation in `ProcessProposal` before finalization [16]. This enables the designer to reject redundant or malformed alerts early while still preserving deterministic behavior, which aligns with the requirement for low-latency actuation in the presence of noisy sensors.

4.5 Non-required features and attack surface

Several features inherited by Quorum are classified as not needed in the requirement analysis. Most notably, Quorum retains the EVM and gas accounting, which are essential for public Ethereum but unnecessary for an SPS Autonomic Manager. These features do not merely sit idle or disabled, they increase runtime overhead

and enlarge the trusted computing base. They also expose the SPS logic to known smart-contract vulnerability classes, such as reentrancy, integer overflow patterns, and access-control errors documented in surveys of Ethereum contract vulnerabilities [20, 21].

CometBFT avoids these inherited features by design. It provides consensus, networking, and a minimal transaction processing pipeline, and it leaves economic incentives, virtual machines, and other overlays to the application. This absence of non-required functionality is in the SPS context a form of alignment, because it keeps the coordination layer narrow and reduces the amount of code that must be trusted or read to make aligned security decisions.

4.6 Security and assurance implications

The SPS manager is a critical component: errors in its logic can lead to false positives, missed attacks, or unsafe countermeasures. When implemented as a smart contract in Quorum, the response logic inherits the risk profile of Ethereum contracts. Extensive analyses show that subtle bugs in contract logic are common and can lead to serious vulnerabilities [20, 21]. Formal verification and defensive coding can mitigate this, but they require specialized expertise and still operate within the constraints of the EVM model.

The SPS coordinator is an critical component that should produce evidence about its own decisions. Quorum's smart-contract model can expose such evidence through immutable logs, yet the opacity of the EVM actions and the complexity of contract execution can make it difficult to relate that evidence to the high-level SPS. A verifier must reason about the intended state machine but also about EVM execution semantics, gas limits, and compiler effects artificially increasing the difficulty of interpreheting the state of the system.

CometBFT enables the same logic to be implemented as native code, where conventional software engineering techniques, static analysis tools, and language-level safety guarantees can be applied. This does not eliminate risk, but it avoids EVM-specific pitfalls and makes it easier to integrate the SPS logic with existing, well-tested security libraries. application code must be deterministic across replicas, or else consensus will diverge.

Another scope that suffer of the same problem is validator management, Quorum's on-chain validator management means that manager decisions are themselves encoded in contracts, which become part of the trusted base. If the contract is flawed or if validator key management is weak, the system's trust boundary can be altered by an attacker. In CometBFT, validator governance is an application policy, this allows integration with existing organizational controls (for example, manual enrollment, hardware-backed keys, or external approval

workflows), but it also requires a careful design to ensure that membership changes are auditable and resistant to compromise.

4.7 Failure modes, recovery, and liveness

Both Quorum and CometBFT are BFT protocols and therefore inherit the classical liveness constraint: if more than one third of validators are Byzantine or offline, consensus stalls.

The GoQuorum documentation makes this explicit for QBFT/IBFT and highlights that block production stops when the threshold is exceeded [12].

cometbftConfig is more compatible with an alert-driven workload because consensus activity becomes a clearer indicator of meaningful events, and idle periods are to be considered normal.

As already stated before timeout behavior further differentiates alignment with SPS latency requirements. In QBFT/IBFT, the protocol advances through rounds with a configurable `requesttimeoutseconds` value and doubles that timeout on consecutive failures, which increases reaction time under partial failure [12]. This backoff is appropriate but can increase delay if chosen incoherently in terms of hardware speed with the specification of the system implementing it. CometBFT follows a similar round-based structure but exposes more control at the application and configuration level, allowing to tune timeouts to the expected alert profile and network conditions.

Recovery behavior depends on how quickly a node can rejoin and reconstruct the shared state. Quorum's full-history model is conservative and strongly consistent, this means that it tends to require more data transfer to resynchronize a new or recovering validator. CometBFT allows the application to define snapshotting or compact state representations as minimal as the designer needs. This trade-off mirrors the overall alignment pattern: Quorum offers stronger defaults but heavier operational cost, while CometBFT offers flexibility at the implementer's cost.

4.8 Final Considerations

As seen in this chapter both the technologies explored in the comparison are a viable choice when building an SPS's autonomic manager since both satisfy all the required properties to implement the system, however since this is an uncommon implementation of a blockchain system, CometBFT's malleability in choosing which specific feature is needed and which not make it fit better in our use-case, and possibly result in better performances

Chapter 5

Implementation Details

The previous chapters established, in a technology-agnostic way, the architecture of a blockchain-based Self-Protecting System (SPS) and the requirement-driven rationale for comparing Quorum and CometBFT as its decision substrate. This chapter answers the question that naturally follows: *how is that abstract model actually implemented, and how can we create a fair comparison between them with real attack traffic?*

The prototype we developed [22] is a fully containerized, docker based system in which a real vulnerable web application is protected by three heterogeneous Intrusion Detection Systems (IDSs) coordinated by a permissioned Byzantine Fault-Tolerant (BFT) ledger. The same project can run in two setting, one on GoQuorum (QBFT) and one on CometBFT, so that every non-trivial component is exercised against both backends under identical conditions. The guiding engineering goals are: - structural fidelity to the architecture of Chapter 3 (every container maps to a model role, trust boundaries enforced by network isolation); - backend interchangeability (identical detection and control planes, so that any measured difference is attributable to the consensus substrate); - deterministic decision logic (identical outputs on every honest replica);

The recurring theme of the chapter is that the two implementations are *semantically equivalent* at the level of the formal state machine of Section 3.3, but *structurally different* in how that state machine is hosted, encoded, and scheduled. This divergence is precisely what the empirical evaluation of Chapter 6 will measure.

5.1 Emulated Network Topology and Trust Boundaries

The whole system is emulated with Kathará [11], a lightweight Docker-based network emulator that preserves the full container isolation primitive on which the threat model of Section 3.6.4 relies: the container boundary is the primary security perimeter, and there is no shared namespace between the manager and the managed system. The topology is partitioned into six isolated segments, each enforcing a distinct trust boundary (Table 5.1, conceptually depicted in Figure 3.1).

Table 5.1: Network segments of the SPS lab

Segment	CIDR	Role / Trust boundary
Untrusted zone	192.168.1.0/24	Internet-facing; hosts the attacker only.
DMZ	10.0.0.0/24	Managed system (Juice Shop); untrusted by assumption.
Management nets	172.16.1–3.0/24	One per IDS↔member/light pair; carries alerts only.
Actuator net	172.16.4.0/24	Carries committed actions to the blind actuator.
Feedback net	10.0.100.0/24	Carries NEGATIVE alerts from the actuator back to the IDSs.
Blockchain net	10.99.0.0/24	Validators, member/light nodes and full node; isolated from the DMZ.

A router container performs NAT between the untrusted zone and the DMZ and, crucially, mirrors all inbound DMZ traffic to the sniffing interfaces of the three IDSs. This mirroring decouples detection from the data path of the application: the IDSs observe a copy of the traffic in promiscuous mode and never interact with the real service, so a compromised IDS cannot affect the availability of the managed system or vice versa. Three properties directly implement the architectural guarantees of Chapter 3: the blockchain network is physically separate from the DMZ, so an attacker who compromises the Juice Shop has no L2 reachability to any validator or member node and the transaction submission API is not exposed to the managed system, each IDS reaches only its own paired blockchain-facing node over a dedicated management network, so a compromised IDS cannot impersonate another IDS's node and the feedback network is a separate channel through which the actuator, and only the actuator, notifies the IDSs that a mitigation has been applied, enabling the loop to close (Section 5.4.4).

5.2 Component Realization

5.2.1 Managed System and Detection Plane

The managed system is the OWASP Juice Shop [23], a deliberately vulnerable Node.js application exposed on port 3000. An attacker container in the untrusted zone drives attack traffic through the router. Four canonical web attack classes are detected in this prototype through a fingerprint, each mapped to a distinct bit of the state vector Sql-Injection, Xss, Path Traversal and a Command Injection. The consolidated state S_t is therefore exactly a n-bit vector anticipated in Section 3.3.3 where n is the number of mapped attack, so 4 in the implementation.

The detection plane is three independent IDSs: Snort 3, Suricata and Zeek, each in its own container and sniffing the mirrored DMZ traffic. Using three heterogeneous engines allow us to respect a diversity principle: a single engine-specific signature bypass cannot poison our detection (and consequently our response) ability. The engines share the same four signatures but express them through three different rule languages: Snort uses `alert tcp` rules with `content` matchers and the `alert_fast` plugin mandatory to force per-packet flushing, Suricata uses equivalent `content/nocase` rules in single-pcap mode, Zeek uses its own signature language with `payload` regular expressions writing to `signatures.log`. All three write human-readable alert lines to a log file. A single Python component, `alert_forwarder.py`, runs on each IDS, tails the log, matches attack keywords, and forwards a JSON alerts to the paired blockchain-facing node using an in-memory queue with ten worker threads and HTTP keep-alive, batching up to one hundred alerts per request. This uniform forwarder makes the detection plane backend-agnostic: the IDSs are unaware of whether their downstream node is a Quorum member or a CometBFT light node, since both expose the same HTTP API, this network setting was specifically engineered to prevent connections or packet dropping under high loads of traffic.

5.2.2 Consensus-and-Decision Plane and the Blind Actuator

The consensus-and-decision plane comprises three validators and four blockchain-facing nodes whose *roles* differ between backends while preserving the same functional split. The three **validators** (`validator0..2`) and the **Member3/FullNode0** form the BFT quorum with $N_v = 4$ the system tolerates $f_v = \lfloor (4-1)/3 \rfloor = 1$ Byzantine validators, the minimum meaningful BFT configuration and sufficient to demonstrate the mechanism on a single service. In Quorum the validators are full `geth` nodes, in CometBFT they run the native `cometbft node` consensus engine plus an accompanying ABCI application hosting the decision state machine.

The four **blockchain-facing nodes** are the API gateways of the manager. Members 0-2 are each paired with one IDS over a management network and ingest its alerts,member 3 (Quorum) or `fullnode0` (CometBFT) is paired with the actuator and forwards committed actions. In Quorum all four are full Ethereum clients (`geth` without `-mine`) that maintain a complete chain replica but do not propose blocks, they expose the IDS-facing HTTP API through a Node.js process (`blockchain_api.js`) that translates HTTP alerts into signed `proposeNewValues` transactions. In CometBFT the three IDS-facing nodes are *light nodes*: their `sps-node` exposes the same HTTP API but submits transactions through CometBFT's `broadcast_tx_async` RPC rather than maintaining a full local replica, and the actuator-facing node is a full node that subscribes to committed

events. This match the light/full node distinction of CometBFT/Tendermint and unloads the validators from serving API traffic. Despite the different internals, the HTTP service exposed to the IDSs is intentionally identical: `POST /alert` accepts a JSON array of {`ids`, `type`, `value`, `timestamp`} objects, `GET /stats`, `GET /alive`, and `POST /config/dedup` expose observability and runtime configuration. This service is the seam that makes the two backends interchangeable from the perspective of the detection and control planes.

The **actuator** is the only component allowed to modify the managed system, and it is by design *blind* (Section 3.2.4): it never queries the Juice Shop's state and never accepts instructions from anyone except an HTTP `POST /action` carrying the action string selected by the state machine. The actuator executes that string on the Juice Shop over a pre-shared SSH key:

the actuator holds no application credentials. After execution it appends a timestamped record to `/var/log/actuator_actions.log` (which the benchmark harness parses for end-to-end latency, //TODO: Ricordarsi di citare questa parte nel capitolo successivo)// and posts a `NEGATIVE ALERT` to each IDS over the feedback network to close the recovery loop (Section 5.4.4).

5.3 The Replicated Decision State Machine

The decision state machine is the concrete instantiation of $\Sigma_t = \langle S_t, R_t, I_t \rangle$ of Section 3.3.3. It is where the two backends are most directly comparable, because the *logic* is identical while the *substrate* is maximally different. This section describes that logic once, then shows how each backend hosts it.

5.3.1 State Representation and the Voting Predicate

The consolidated state S_t is a 4-bit vector in canonical order `[SQL_INJECTION, XSS_ATTACK, PATH_TRAVERSAL, COMMAND_INJECTION]`, `1` means an attack has been confirmed by voting, `0` means it has not. The response relation R_t is a state–action map from the $2^4 - 1 = 15$ non-trivial bit-strings to mitigation commands. Because multiple attacks may be active simultaneously, each base attack maps to an unix-command action and composite states map to the concatenation of the set bits' actions with `&&`. It is keyed by a cryptographic hash of the state string `keccak256` in the EVM, `SHA-256` in ABCI Rust, When the consolidated state changes, the machine hashes it and, if a matching entry exists, emits the associated action.

When an agent submits a proposal, a tally is decremented and the new one incremented, if the agent had not voted, it is recorded and the tally incremented. The consolidation predicate then requires the tally for the new value to exceed

$\lfloor N_d/2 \rfloor$ with $N_d = 3$, i.e. at least two independent IDSs must agree before the state flips, see the majority threshold $q_j = 2$ of Section 3.3.2.

5.3.2 Quorum Realization: The `IDS.sol` Smart Contract

In Quorum the state machine is a Solidity contract, `IDS.sol`, compiled with `solc` and deployed on validators at startup. Each parameter is a `Parameter_Status` struct with the current value, per-agent proposed value, per-value vote count, and a `hasVoted` flag; the agents are the addresses of members 0–2, injected at configuration time by templating the source (Section 5.8.3). When enough nodes votes reaching the threshold, the contract concatenates the current values, hashes them with `keccak256`, looks up the action, and emits an `ActionRequired(address selectedAgent, string actionToApply)` event. The selected agent is `keccak256(block.timestamp, block.number) % agents.length`: this is per block deterministic because all validators agree on `block.timestamp` and `block.number` for the same block, it is important to point out the deterministic nature of used params, since a non-deterministic source would cause replicas to diverge causing the blockchain's reliability to fail.

5.3.3 CometBFT Realization: The Rust ABCI Application

CometBFT state machine's implementation is a native Rust application, `sps-node`, which uses the Application BlockChain Interface (ABCI) [16], CometBFT acts purely as the consensus and networking black box: it orders transactions, replicates the block log, and guarantees BFT finality, but knows nothing about IDSs, attacks, or actions. All application semantics live in the application, whose relevant ABCI handlers are: `CheckTx`, invoked on mempool admission, `DeliverTx`, invoked in deterministic block order for every committed transaction, which decodes the transaction, applies it to the ledger, and, if the transition triggers an action, publishes it on an in-process broadcast channel.

The transaction payload is a `VoteTx` struct serialized with `bincode`, a compact binary codec faster in performances than regular json. the resulting transaction is a few dozen bytes, keeping mempool pressure and block size minimal even under pressure. The action produced is pushed onto a broadcast channel to which the actuator-facing full node subscribes and forwards each `ActionEvent` to the actuator over HTTP. This in-process eventing is the CometBFT analogue of the Quorum event log and is significantly cheaper because it requires no polling of block history.

5.4 The End-to-End Alert Response Loop

This section traces a single alert through the full MAPE-K cycle. The sequence is identical for both backends up to the transaction encoding, which is the intended guarantee of backend interoperability.

5.4.1 Alert forwarding.

When an IDS matches a signature it writes a log line, classifies the line into one of the four attack types (or a `NEGATIVE ALERT` recovery), and enqueues a JSON payload {`ids`, `type`, `value`, `timestamp`} with `value=1` for an attack and `0` for recovery. Ten worker threads drain the queue in batches of up to one hundred and post each batch to `http://<member_ip>:3000/alert` over a persistent connection. Request batching is used for high network stress situation, a single attack packet can trigger an alert on all three IDSs within milliseconds, and a burst can produce thousands of alerts per second, so per-alert HTTP requests could saturate the member node's socket table before consensus becomes the bottleneck.

5.4.2 From HTTP alert to committed transaction.

The member/light node normalizes the batch into a transaction. In Quorum the Node.js API coalesces it into one `proposeNewValues` call and enqueues it in a bounded queue drained by concurrent web3 senders with nonce resynchronization (Section **??**). In CometBFT the Rust API encodes it into a `VoteTx`, a background worker submits it through `broadcast_tx_async`. In both cases the transaction enters the mempool, is gossiped, ordered into a block by the BFT protocol, and applied to the state machine where the voting predicate is evaluated.

5.4.3 Event delivery and actuation.

When the consolidated state transitions to a mapped action, an `ActionRequired` event is emitted. In Quorum this is an EVM log entry. In CometBFT the action is delivered through the in-process `broadcast` channel with no polling, since it is produced synchronously inside `DeliverTx`. In both cases the actuator receives the same JSON payload and executes it over SSH

5.4.4 Feedback and loop closure.

After mitigating, the actuator posts a `NEGATIVE ALERT` carrying the mitigated attack type to each IDS over the feedback network. A tiny `ids_feedback_server.py` on each IDS appends a `NEGATIVE ALERT: <type>` line to the very log file that

`alert_forwarder.py` is tailing; the forwarder interprets it as a recovery proposal with `value=0` and forwards it. Once two of the three IDSs report recovery, the predicate flips the bit back to `0`, if the new state no longer maps to any action, no further actuation fires and the loop is closed.

5.5 The Deduplication Middleware

A sustained Cyber attack such as a timed SQL-Injection attempt can produce hundreds of alerts per second, and three heterogeneous IDSs can each emit a matching alert within the same few milliseconds, since a BFT block can takes on the order of second to commit, the mempool would fill with thousands of redundant transactions describing the same transition long before the first one commits, stalling consensus and inflating latency by orders of magnitude. The deduplication middleware prevents this and is a core contribution of the thesis. It is an API-level filter that drops redundant alerts in the same block before the mempool, since those redudant alert would already only be registered as a single response, this filter save us many CPU cycles and prevent indefinitely growing mempool.

5.6 Quorum Filter

The Quorum API maintains `lastVotedState` (the value last sent per parameter) and `batchBuffer` (pending alerts since the last flush). Incoming alerts update the buffer, if the value matches `lastVotedState` the alert is acknowledged as `deduplicated` and never reaches the contract. The buffer is flushed into a single `proposeNewValues` transaction when either a new block is observed. On each confirmed block `lastVotedState` is reset, so at most one transaction per parameter per block is ever submitted. This block-aligned window is natural for a fixed-interval backend, and a single `proposeNewValues` call carries all four parameters, amortizing per-transaction overhead.

5.7 CometBFT Filter

The CometBFT API similarly to the Quorum's maintains a `seen_types` set of attack types already forwarded in the current block. The first alert of a given type, any subsequent one is dropped locally with a `DEDUP DROP` log, without waiting for BFT confirmation—because the BFT round is slower ((≈0,1s)) than the alert inter-arrival time (≈ms), waiting for on-chain confirmation would defeat the purpose.

5.8 Backend-Specific settings

5.8.1 Quorum: BlockPeriod and Gas Fees

GoQuorum inherits the full Ethereum stack, and aligning it with the SPS requirements requires explicitly disabling features that are not needed in a permissioned closed deployment. The genesis sets `blockperiodseconds=1`. The gas limit is raised to `0xFFFFFFFFF` and the gas price set to zero on every transaction, neutralizing the gas/fee machinery,this doesn't prevent to waste CPU cycles in it, and also does not remove it from the trusted computing base. `maxCodeSize` and `txnSizeLimit` are capped at 256 KB; the network runs with `-nodiscover` and a static full-mesh `permissioned-nodes.json`, disabling dynamic peer discovery. The transaction pool is sized (`globalslots=250000`) to absorb huge burst at a RAM cost.

5.8.2 CometBFT: ABCI and Timeouts Tuning

The CometBFT backend is configured to exploit the event-driven block production identified as a native advantage. The consensus `config.toml` sets `create_empty_blocks=false` (don't provide a new block without pending transactions) and `skip_timeout_commit=true` (immediate advance to the next round after commit). The propose and vote timeouts are set at (1 s with 200 ms deltas) The mempool holds 200 000 entries with a 256 KB per-transaction ceiling.

The `sps-node` binary is built as a statically linked musl target through a multi-stage Docker image, so the runtime container carries only the binary, CometBFT, and a minimal Python runtime for the shared forwarders; no Rust toolchain is needed in the lab. A single binary serves three roles—`validator`, `agent` (light), `fullnode`—selected by `sps_config.toml`, and the action forwarder task is spawned only on full nodes and validators. The ABCI server runs on a dedicated OS thread bound to `127.0.0.1:26658`, while the HTTP API and the submission worker run on the tokio runtime, so consensus-critical decoding never contends with API IO on the same thread.

5.8.3 Configuration Generation and SSH Keys

Both backends rely on a Python configuration generator run on the host before the lab starts, so no node ever generates keys or peer lists at runtime. For Quorum, `generate_blockchain_config.py` drives the official `quorum-genesis-tool` inside a throwaway boot lab to produce keystores, the QBFT genesis, and enode identities, then assembles a full-mesh `static-nodes.json` with the correct lab IPs by substituting member addresses and the parameter list from `IDS.sol`. For

CometBFT, `generate_cometbft_config.py` derives each node's Tendermint identifier as the first 20 bytes of `SHA-256(pubkey)`, writes the validator key files, and emits a genesis with the three validators at equal voting power, it also renders the application `sps_config.toml` and the tuned CometBFT `config.toml`. Both generators produce the actuator's Ed25519 SSH keypair once and share it through the lab volume. A small startup handshake ensures deterministic boot ordering: in CometBFT, `fullnode0` waits for a genesis published by `validator0` before starting, so all nodes agree on the initial validator set.

Chapter 6

Experiments and results

6.1 Evaluation Methodology

6.2 Performance Analysis

6.2.1 Effectiveness of the Deduplication Mechanism

6.2.2 End-to-End Latency under Low Traffic

6.2.3 Transaction Throughput under Increasing Load

Chapter 7

Conclusions and Future Work

7.1 Summary of Research Findings

7.2 Limitations of the Study

7.3 Future Directions

Bibliography

[1] Eric Yuan, Naeem Esfahani, and Sam Malek. "A Systematic Survey of Self-Protecting Software Systems". In: *ACM Trans. Auton. Adapt. Syst.* 8.4 (Jan. 2014). ISSN: 1556-4665. DOI: 10.1145/2555611. URL: https://doi.org/10.1145/2555611.

[2] Tommaso Caiazzi et al. "A Novel Architecture for Cyber-Resilient Self-Protecting Systems Based on Blockchain". In: *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. 2025, pp. 1–8. DOI: 10.1109/COMPSAC65507.2025.00010.

[3] Jeffrey O Kephart and David M Chess. "The vision of autonomic computing". In: *Computer* 36.1 (2003), pp. 41–50.

[4] Eric Yuan, Naeem Esfahani, and Sam Malek. "A systematic survey of self-protecting software systems". In: *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 8.4 (2014), pp. 1–41.

[5] Paolo Arcaini, Elvinia Riccobene, and Patrizia Scandurra. "Modeling and Analyzing MAPE-K Feedback Loops for Self-Adaptation". In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2015, pp. 13–23. DOI: 10.1109/SEAMS.2015.10.

[6] David M Chess, Charles C Palmer, and Steve R White. "Security in an autonomic computing environment". In: *IBM Systems journal* 42.1 (2003), pp. 107–118.

[7] Irdin Pekaric et al. "A systematic review on security and safety of self-adaptive systems". In: *Journal of Systems and Software* 203 (2023), p. 111716.

[8] Xin Wang et al. "Bft in blockchains: From protocols to use cases". In: *ACM Computing Surveys (CSUR)* 54.10s (2022), pp. 1–37.

[9] Vitalik Buterin et al. "Ethereum white paper". In: *GitHub repository* 1 (2013), pp. 22–23.

[10] Simone Albero et al. "Which Blockchain Technology Best Supports Self-Protecting Systems? A comparison between Quorum and CometBFT". In: *Proceedings of the 8th Distributed Ledger Technology Workshop (DLT 2026)*. In press. Pula, Italy: CEUR-WS.org, June 2026.

[11] M. Scazzariello, L. Ariemma, and T. Caiazzi. "Kathará: A Lightweight Network Emulation System". In: *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*. 2020.

[12] ConsenSys. *QBFT | ConsenSys GoQuorum*. [Online; accessed 2025-02-26]. Nov. 2023. URL: https://docs.goquorum.consensys.io/configure-and-manage/configure/consensus-protocols/qbft.

[13] Henrique Moniz. "The Istanbul BFT consensus algorithm". In: *arXiv preprint arXiv:2002.03613* (2020).

[14] Ethan Buchman, Jae Kwon, and Zarko Milosevic. "The latest gossip on BFT consensus". In: *arXiv preprint arXiv:1807.04938* (2018).

[15] CometBFT Developers. *CometBFT*. https://cometbft.com. Accessed April 2026. 2025.

[16] Cosmos SDK Documentation. *ABCI Overview*. Accessed May 2026. 2026. URL: https://docs.cosmos.network/sdk/latest/guides/abci/abci.md.

[17] CometBFT Developers. *CometBFT Configuration*. Accessed May 2026. 2026. URL: https://github.com/cometbft/cometbft/blob/main/config/config.go.

[18] Gianmaria Del Monte, Diego Pennino, and Maurizio Pizzonia. "Scaling blockchains without giving up decentralization and security: A solution to the blockchain scalability trilemma". In: *Proceedings of the 3rd Workshop on Cryptocurrencies and Blockchains for Distributed Systems*. 2020, pp. 71–76.

[19] Stefano Iannucci et al. "A model-integrated approach to designing self-protecting systems". In: *IEEE Transactions on Software Engineering* 46.12 (2018), pp. 1380–1392.

[20] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. "A survey of attacks on ethereum smart contracts (sok)". In: *International conference on principles of security and trust*. Springer. 2017, pp. 164–186.

[21] Satpal Singh Kushwaha et al. "Systematic review of security vulnerabilities in ethereum blockchain smart contract". In: *IEEE Access* 10 (2022), pp. 6605–6621.

[22] stigmatalgia. *SelfProtectingSystem: A Novel Architecture for Cyber-Resilient SPS Based on Blockchain*. https://github.com/stigmatalgia/SelfProtectingSystem. Accessed: April 2026. 2025.

[23] OWASP Foundation. *OWASP Juice Shop*. https://owasp.org/www-project-juice-shop/. Accessed: April 2026. 2024.